Документация к файлу `mysterious_file.py`

Функция 1

def bought_products(money, price_per_item, items_count):

- Вычитает сумму за покупку одинаковых товаров из общей суммы денег у пользователя

Параметры

1. `money` (int): Число денег у пользовавтеля
2. `price_per_item` (float): Цена одного товара
3. `items_count` (int): Число одинаковых товаров

Возвращает:

1. `int` или `float` : Оставшиеся деньги у пользователя
2. `str` : В противном случае "Недостаточно денег", если денег не хватает

Примеры:

```
# 1
bought_products(100, 10, 5) # 50
```

```
# 2
bought_products(100, 100, 2) # "Недостаточно денег"
```

Функция 2

is_in_list(day):

- Делает проверку на наличие элемента в заранее заготовленном списке, состоящим из дней недель

Параметры

1. `day` (str): Элемент, наличие которого мы хотим проверить (День недели)
2. `arr = ["сб", "вс", "суббота", "воскресенье"]` (list): Массив дней недели

Возвращает:

1. `bool` :
    - `True` - Элемент есть в массиве
    - `False` - Элемента в массиве нет

Примеры:

```
# 1
is_in_list("Суббота") # True
```

```
# 2
is_in_list("C") # False
```

Функция 3

def is_even(parametr):

- Функция, делающая проверку чётное ли число

Параметры

1. `parametr` (int): Элемент, который мы хотим проверить

Возвращает:

1. `bool` :
    - `True` - Число кратно 2
    - `False` - Число не кратно 2

Примеры:

```
# 1
is_even(2) # True
```

```
# 2
is_even(3) # False
```

Функция 4

multiples_of_3(prm1, prm2, step = 1):

- Функция возвращает список чисел, кратных 3

Параметры

1. `prm1` (int): Начало диапозона перебора индексов
2. `prm2` (int): Конец диапозона перебора индексов
3. `step` (int) - Шаг перебора индексов
    - Если при вызове ф-ции шаг(step) **не обозначаем =>** `step = 1`
    - Если при вызове ф-ции шаг(step) **обозначаем =>** `step` = <введённое число>

Возвращает:

1. `list` : Список чисел, которые делятся на 3

Примеры:

```
# 1
multiples_of_3(0, 13) # [0, 3, 6, 9]
```

```
# 2
multiples_of_3(0, 13, 2) # [0, 6, 12]
```

```
# 3
multiples_of_3(0, 4, 1) # [0, 3]
```

Функция 5

sentiment_score(txt, pos, neg):

- Функция возвращает коэффицент (оценку) того, на сколько сильно текст был эмоционально окрашен

Параметры

1. `txt` (str): Строка, которую будет оценивать функция
2. `pos` (list): Список положительных слов (прилагательных)
3. `neg` (list) - Список негативных слов (прилагательных)

Возвращает:

1. `float` : Коэффицент эмоциональной окраски текста, округлённый до десятых

Примеры:

```
# 1
sentiment_score("I love Python!", ["love", "like", "enjoy", "adore", "appreciate"], ["hate",
```

```
# 2
sentiment_score("love like enjoy", ["love", "like", "enjoy"], ["hate"]) # 3.0
```

```
# 3
sentiment_score("Do I love! Python", ["love"], ["hate"]) # 0.0
```

Функция 6

transform_into_binary(num, count = 0):

- Функция делает перевод числа из десятичной системы счистления в двоичную и выводит результат. В текущей версии переменная count не выводится

Параметры

1. `num` (int): Число в десятичной системы счистления
2. `count` (int): Количество итераций

Возвращает:

1. `str` : Строка из символов входящего числа в двоичной системе счистления

Примеры:

```
# 1
transform_into_binary(10) # '0101'
```

```
# 2
transform_into_binary(10, 3) # ('0101', 7)
```

```
# 3
transform_into_binary(12) # ('1100', 4)
```

```
# 4
transform_into_binary(12, 23) # ('1100', 27)
```

История изменений ветки doc_561160

1. `785a7c6` *Сообщение:* "add(docs): add <docs> folder for functions description and fixing <mysterious_file.py>"
2. `f9a4f55` *Сообщение:* "update(mysterious_file): add true descriptions for functions"
3. `41c7cc1` *Сообщение:* "add(docs): add <README.md> file with descriptions for functions, without code"
4. `43b0045` *Сообщение:* "update(docs): add commit logs in <README.md>"
5. `0360688` *Сообщение:* "update(docs): add commit logs in <README.md>"
6. `44e2758` *Сообщение:* "update(mysterious_file): add new comments"
7. `4577f14` *Сообщение:* "add(docs): add documentation folder for <mysterious_file.py>"
8. `5164d21` *Сообщение:* "fix(): fixed rectangle, triangle and my_main_561160"